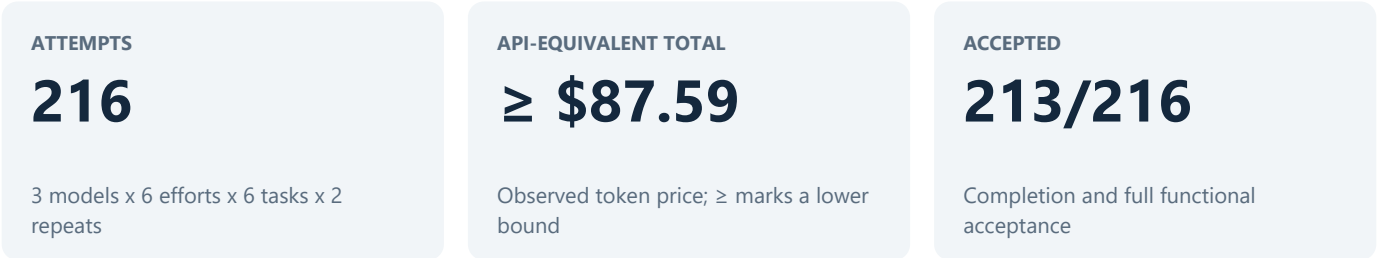


Sol coding benchmark

Six programming tasks. Three models. Six reasoning effort levels.

Sol 5.6, Sol 6 and Sol 6.1 at Low, Medium, High, Xhigh, Max and Ultra. Two independent attempts per task, Standard speed.



What the results show

Best measured cost with full acceptance: Sol 6.1 / low: \$0.101 per attempt, 161.7 seconds, 12/12 accepted. This is also the fastest setting with full acceptance.

Complex tasks: 71/72 accepted; 1115/1116 hidden checks passed. One Sol 6 Low optimizer attempt exceeded the 8-second scale limit. Every optimizer small-case oracle and all MVCC checks passed.

Broader behavioral evidence: 920/922 expanded checks passed on completed solutions for the first four tasks. Two Low cache solutions fail a valid huge-integer TTL boundary. 214/216 attempts completed; the two earlier noncompletions remain included in delivery and cost accounting.

Own-test evidence: 175/214 completed candidates have a usable score, including zero for no discovered added tests. 29 have no recognized added tests. The two new tasks have 55/72 usable scores. Manual rubric scoring remains incomplete.

Test-strength separation: Sol 6.1 MVCC added tests detect all six fixed defects in both repetitions at High, Xhigh, Max and Ultra. Optimizer test-strength evidence is often inconclusive, so this benefit is specific to the MVCC defect set.

How to interpret the comparison

The lower-cost settings already solve nearly all tested behavior. Higher effort generally costs more and takes longer, with little additional correctness separation here. The complex tasks still show a strong score ceiling: this evidence supports a workload-specific comparison, without a reliable ranking of general intelligence.

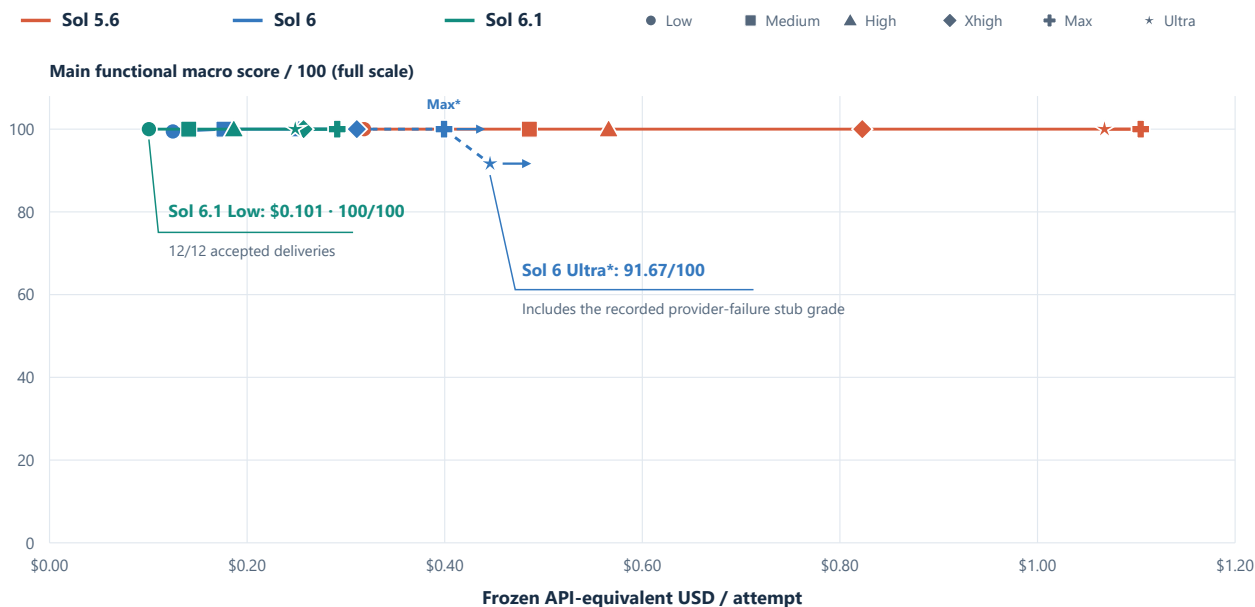
USD is a counterfactual Standard API token estimate for observed Codex usage, not an invoice. Partial usage creates lower bounds. The primary charts place API-equivalent dollars on X and functional quality on Y; effort points are connected within each model.

Cost versus functional quality

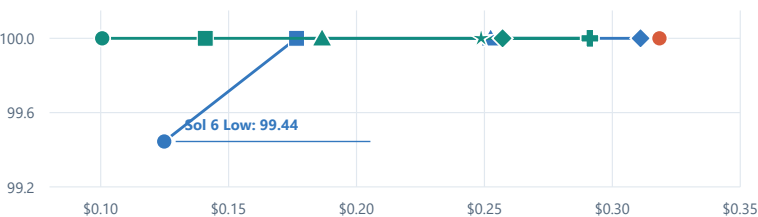
X: mean API-equivalent USD per attempt. Y: equal six-task functional macro score, out of 100. Each setting contains twelve attempts. Higher quality for less money occupies the upper-left region.

What does more money buy on these coding tasks?

Cost per attempt versus main functional score · 6 tasks × 2 repetitions per setting



Near-ceiling detail: USD \$0.08–\$0.35; score 99.2–100.15



* Sol 6 Max / Ultra: observed cost lower bounds. Rightward arrows mean the complete cost is unknown.
Lines follow effort order, not interpolation. Upper-left means less money and a higher measured functional score.

The full 0-100 scale makes the score ceiling visible. Most settings score 100; large spending differences buy little main-score separation in this suite. Effort symbols and colors identify every point; the complete table follows.

Lines connect effort order within each model. Asterisks, dashed links and rightward arrows mark observed cost lower bounds, because the complete cost of Sol 6 Max and Ultra is unknown. They are not quality uncertainty bars.

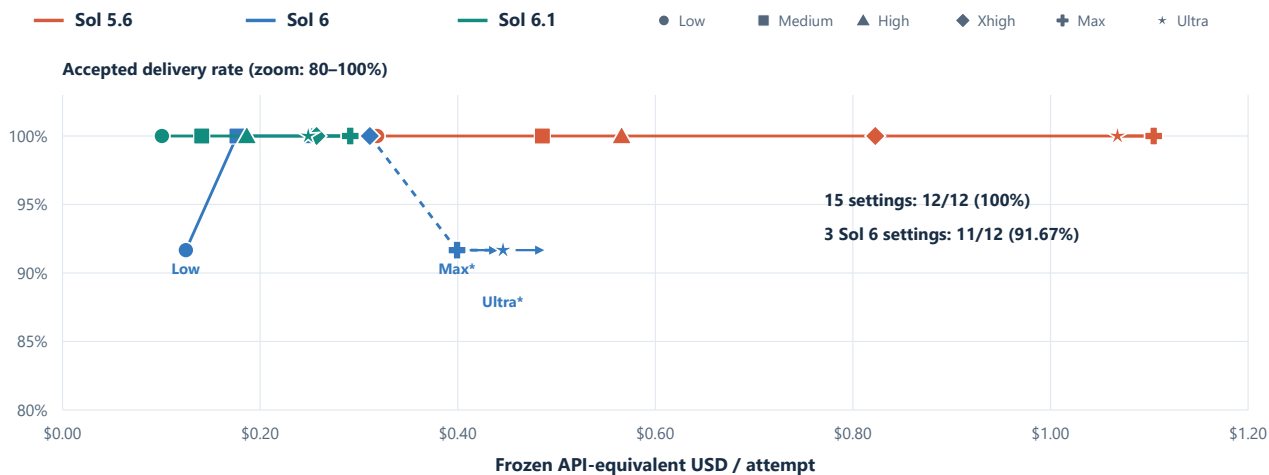
This is observable contract behavior, not a completed overall code-quality grade. The timeout snapshot passes functional checks but is not an accepted delivery. The manual design rubric remains pending.

Cost versus accepted delivery

X: mean USD per attempt, including unsuccessful spending. Y: completed deliveries that pass main acceptance, as a percentage of twelve attempts.

The passing snapshot and the completed delivery differ

Supplementary outcome measure · accepted = completed status and accepted main grade



The Sol 6 Max service snapshot scores 100, but its delivery timed out. It therefore reduces this rate, not its functional score.

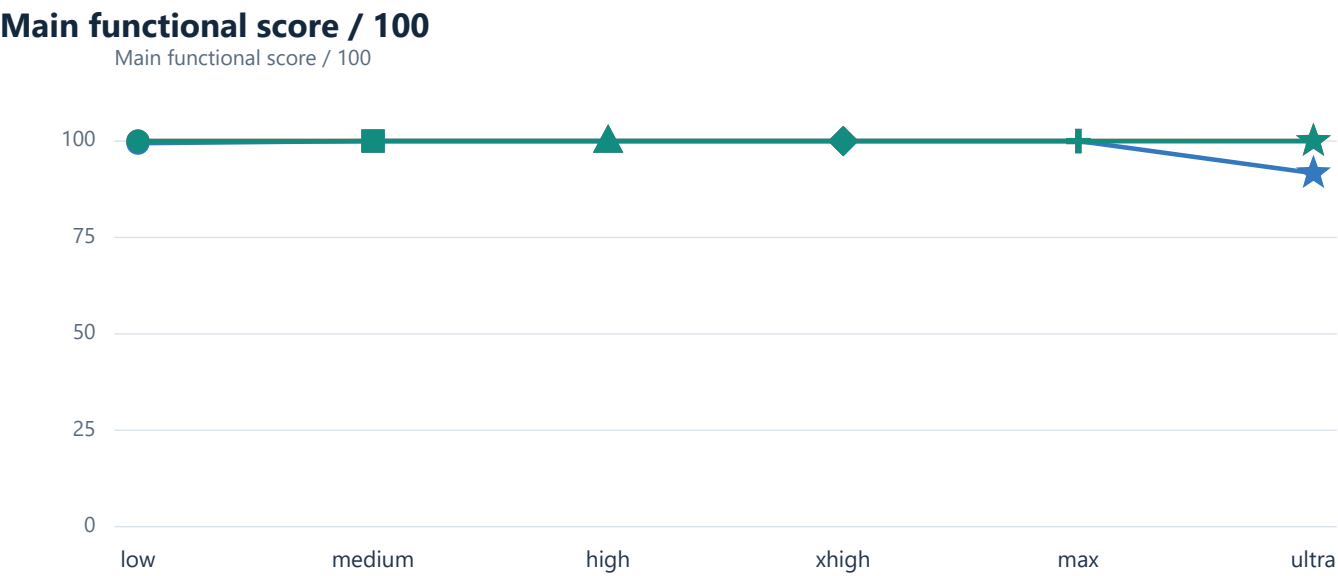
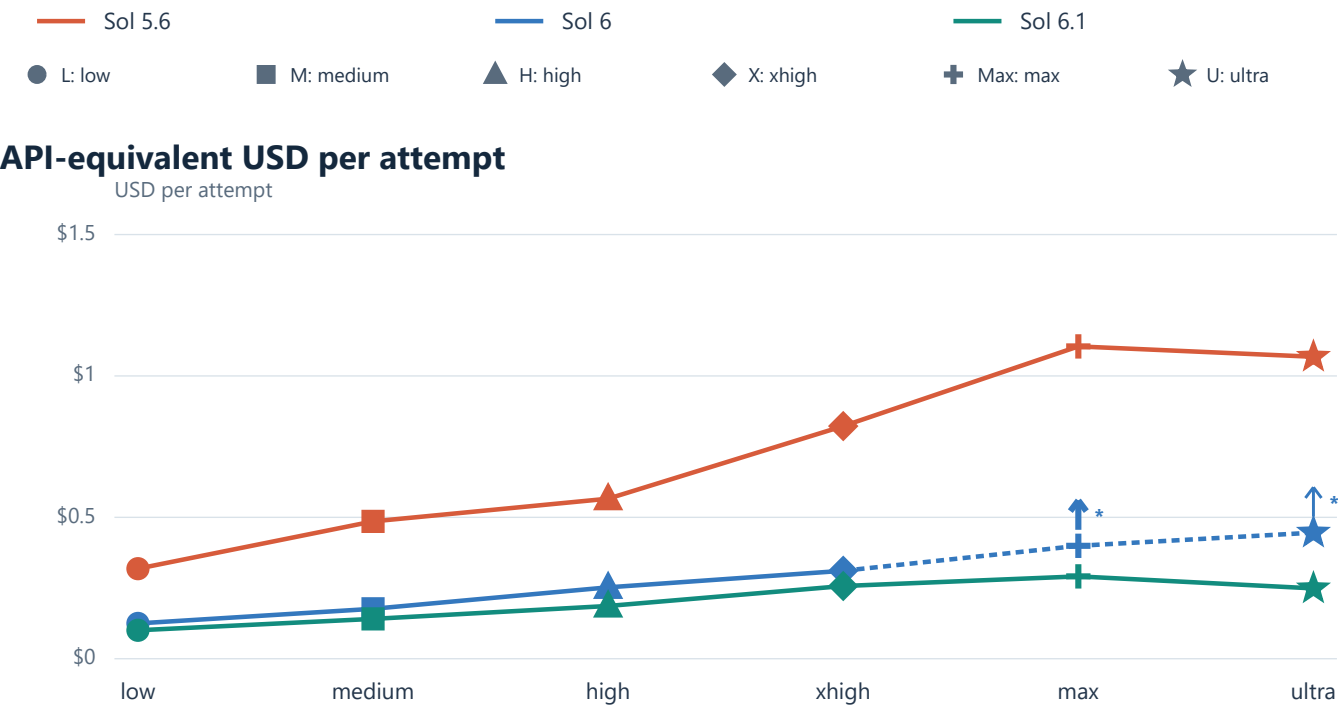
* Partial cost telemetry: rightward arrows mark lower bounds. All attempted work stays in the cost denominator.

Fifteen settings deliver 12/12 accepted attempts. Sol 6 Low, Max and Ultra deliver 11/12 (91.67%) for three different reasons: optimizer scaling, unfinished service delivery and provider capacity. These observations do not estimate a production failure probability.

Acceptance adds completed delivery and protected/public-test gates to functional behavior. This separate chart prevents a passing snapshot after timeout from appearing equivalent to a completed accepted delivery. Only two repetitions per task were collected.

Effort curves: money and quality

Follow each model across six settings. Dashed USD segments use partial observations.



Higher effort is a setting, not a guarantee of higher correctness. Compare these premiums with the behavior and acceptance tables.

Dollars, quality and delivery

Twelve attempts per setting. USD / accepted includes spending on unsuccessful attempts.

Model / effort	USD / attempt	Functional / 100	Accepted	USD / accepted	Mean time, s
Sol 5.6 / low	\$0.318	100.00	12/12	\$0.318	194.2
Sol 5.6 / medium	\$0.486	100.00	12/12	\$0.486	282.8
Sol 5.6 / high	\$0.566	100.00	12/12	\$0.566	350.1
Sol 5.6 / xhigh	\$0.823	100.00	12/12	\$0.823	483.7
Sol 5.6 / max	\$1.104	100.00	12/12	\$1.104	622.6
Sol 5.6 / ultra	\$1.068	100.00	12/12	\$1.068	626.8
Sol 6 / low	\$0.125	99.44	11/12	\$0.136	158.7
Sol 6 / medium	\$0.177	100.00	12/12	\$0.177	226.7
Sol 6 / high	\$0.252	100.00	12/12	\$0.252	300.2
Sol 6 / xhigh	\$0.311	100.00	12/12	\$0.311	402.6
Sol 6 / max	≥ \$0.399*	100.00	11/12	>= \$0.435*	814.6
Sol 6 / ultra	≥ \$0.445*	91.67	11/12	>= \$0.486*	626.4
Sol 6.1 / low	\$0.101	100.00	12/12	\$0.101	161.7
Sol 6.1 / medium	\$0.141	100.00	12/12	\$0.141	246.7
Sol 6.1 / high	\$0.187	100.00	12/12	\$0.187	453.1
Sol 6.1 / xhigh	\$0.257	100.00	12/12	\$0.257	588.6
Sol 6.1 / max	\$0.291	100.00	12/12	\$0.291	676.8
Sol 6.1 / ultra	\$0.249	100.00	12/12	\$0.249	577.1

Sol 6 / Max has a 40-minute reservation-service timeout; its saved code passes the functional and expanded checks. Sol 6 / Ultra has a provider capacity failure on the interval task. Sol 6 / Low completes every attempt but misses one optimizer scale check. These are distinct outcomes.

Displayed >= costs are observed lower bounds, rounded down. Accepted-cost totals and exact token estimates remain in the public data; incomplete final spending is not assigned zero.

What does extra effort buy?

Multiples use the same model at Low as the baseline. Functional quality is an equal six-task macro average.

Model / effort	Cost multiple	Functional / 100	Accepted	Time multiple
Sol 5.6 / low	1.00	100.00	12/12	1.00
Sol 5.6 / medium	1.53	100.00	12/12	1.46
Sol 5.6 / high	1.78	100.00	12/12	1.80
Sol 5.6 / xhigh	2.58	100.00	12/12	2.49
Sol 5.6 / max	3.47	100.00	12/12	3.21
Sol 5.6 / ultra	3.35	100.00	12/12	3.23
Sol 6 / low	1.00	99.44	11/12	1.00
Sol 6 / medium	1.41	100.00	12/12	1.43
Sol 6 / high	2.02	100.00	12/12	1.89
Sol 6 / xhigh	2.49	100.00	12/12	2.54
Sol 6 / max	N/A	100.00	11/12	5.13
Sol 6 / ultra	N/A	91.67	11/12	3.95
Sol 6.1 / low	1.00	100.00	12/12	1.00
Sol 6.1 / medium	1.40	100.00	12/12	1.53
Sol 6.1 / high	1.85	100.00	12/12	2.80
Sol 6.1 / xhigh	2.56	100.00	12/12	3.64
Sol 6.1 / max	2.90	100.00	12/12	4.19
Sol 6.1 / ultra	2.47	100.00	12/12	3.57

The main score averages the two repetitions within each task, then gives each of six tasks equal weight. It includes retained snapshot grades; acceptance also requires completed delivery. The capacity-failed stub scores zero; the timeout snapshot has passing code but is not accepted delivery.

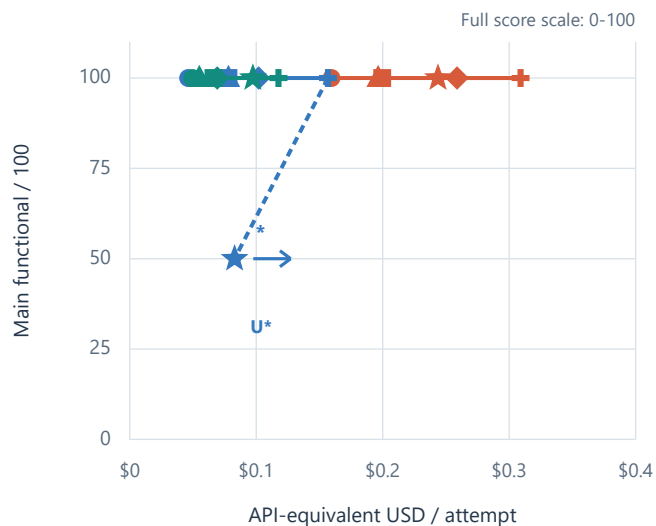
Do not read a small score difference as a statistically established effect. Each model/effort/task has only two observations.

Workload cost and quality: core tasks

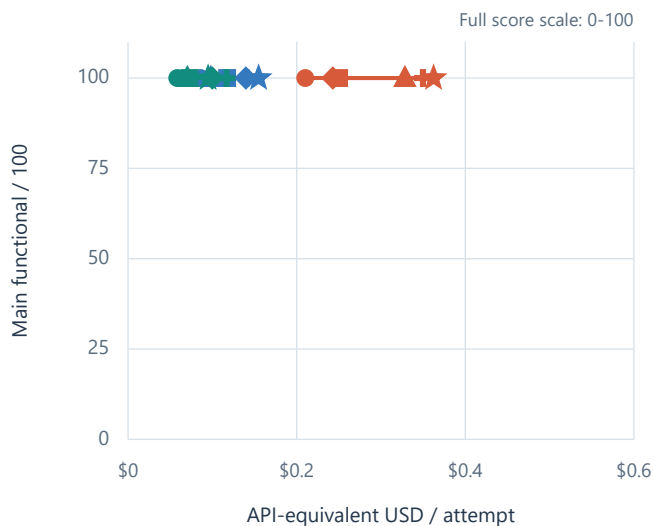
Each point averages two attempts: USD on X, task functional score on Y. Dollar ranges differ; every quality axis uses the full 0-100 scale.



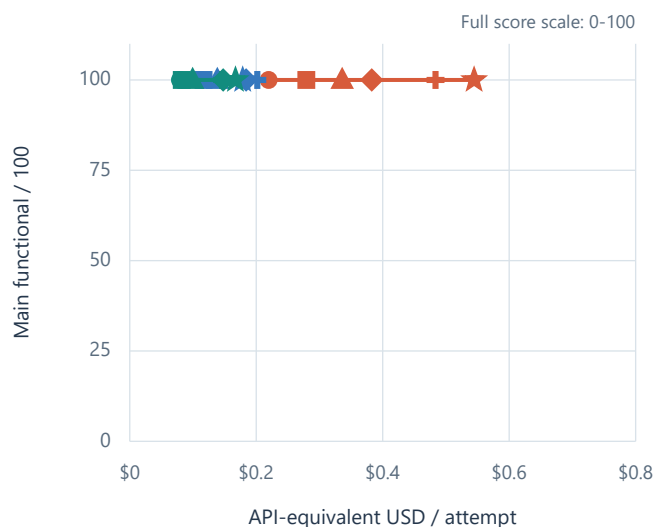
Intervals



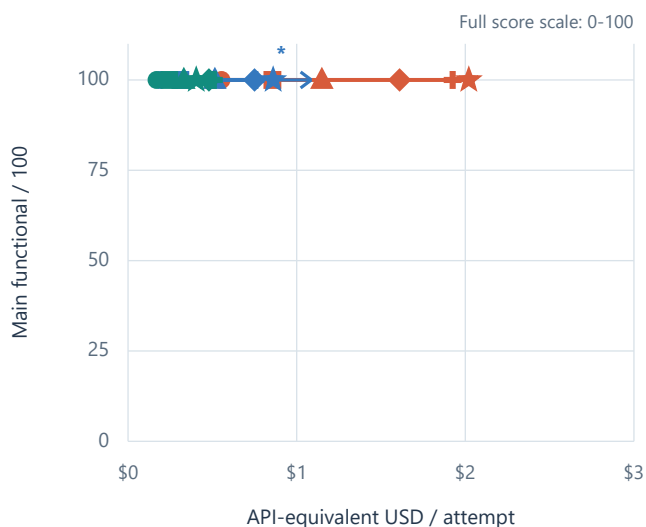
TTL/LRU cache



Concurrent DAG



SQLite service



Intervals test inclusive union and purity; the cache tests TTL/LRU semantics; the DAG tests concurrency and failure propagation; the service combines SQLite transactions, HTTP and CLI behavior.

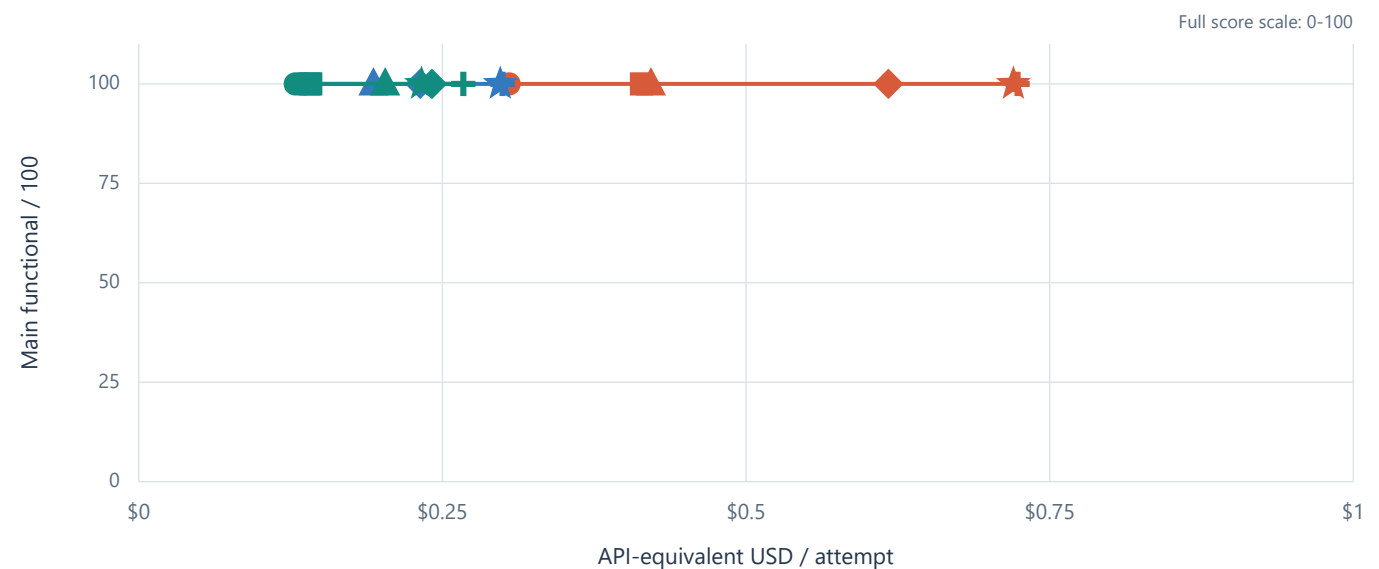
Workload cost and quality: complex tasks

Algorithmic optimization and interacting transaction invariants add complementary complexity.



Signed values, transitive prerequisites, one-sided exclusions, mandatory projects, up to three resource budgets, exact optimality and cost/ID tie-breaking. Structured cases contain 32 projects.

Serializable MVCC store with recovery



Snapshot reads, staged writes, conservative optimistic validation, missing-key/range reads, savepoints, retained read history and checkpoint/log replay. Calls use deterministic in-memory interleavings.

Task-level dollars and quality

Each cell shows mean API USD above mean functional score / 100, across two repetitions.

Model / effort	Intervals	Cache	DAG	Service	Optimizer	MVCC
Sol 5.6 / low	\$0.160 100.00 / 100	\$0.210 100.00 / 100	\$0.220 100.00 / 100	\$0.554 100.00 / 100	\$0.462 100.00 / 100	\$0.305 100.00 / 100
Sol 5.6 / medium	\$0.200 100.00 / 100	\$0.250 100.00 / 100	\$0.279 100.00 / 100	\$0.856 100.00 / 100	\$0.915 100.00 / 100	\$0.414 100.00 / 100
Sol 5.6 / high	\$0.196 100.00 / 100	\$0.328 100.00 / 100	\$0.336 100.00 / 100	\$1.149 100.00 / 100	\$0.964 100.00 / 100	\$0.422 100.00 / 100
Sol 5.6 / xhigh	\$0.259 100.00 / 100	\$0.243 100.00 / 100	\$0.383 100.00 / 100	\$1.610 100.00 / 100	\$1.824 100.00 / 100	\$0.617 100.00 / 100
Sol 5.6 / max	\$0.309 100.00 / 100	\$0.350 100.00 / 100	\$0.484 100.00 / 100	\$1.925 100.00 / 100	\$2.836 100.00 / 100	\$0.724 100.00 / 100
Sol 5.6 / ultra	\$0.244 100.00 / 100	\$0.362 100.00 / 100	\$0.545 100.00 / 100	\$2.021 100.00 / 100	\$2.515 100.00 / 100	\$0.720 100.00 / 100
Sol 6 / low	\$0.046 100.00 / 100	\$0.086 100.00 / 100	\$0.113 100.00 / 100	\$0.191 100.00 / 100	\$0.176 96.67 / 100	\$0.137 100.00 / 100
Sol 6 / medium	\$0.078 100.00 / 100	\$0.118 100.00 / 100	\$0.117 100.00 / 100	\$0.310 100.00 / 100	\$0.297 100.00 / 100	\$0.140 100.00 / 100
Sol 6 / high	\$0.078 100.00 / 100	\$0.098 100.00 / 100	\$0.138 100.00 / 100	\$0.515 100.00 / 100	\$0.492 100.00 / 100	\$0.193 100.00 / 100
Sol 6 / xhigh	\$0.102 100.00 / 100	\$0.140 100.00 / 100	\$0.184 100.00 / 100	\$0.750 100.00 / 100	\$0.459 100.00 / 100	\$0.232 100.00 / 100
Sol 6 / max	\$0.157 100.00 / 100	\$0.140 100.00 / 100	\$0.202 100.00 / 100	≥ \$0.754* 100.00 / 100	\$0.843 100.00 / 100	\$0.300 100.00 / 100
Sol 6 / ultra	≥ \$0.082* 50.00 / 100	\$0.155 100.00 / 100	\$0.179 100.00 / 100	\$0.861 100.00 / 100	\$1.101 100.00 / 100	\$0.298 100.00 / 100
Sol 6.1 / low	\$0.050 100.00 / 100	\$0.058 100.00 / 100	\$0.079 100.00 / 100	\$0.169 100.00 / 100	\$0.118 100.00 / 100	\$0.129 100.00 / 100
Sol 6.1 / medium	\$0.066 100.00 / 100	\$0.073 100.00 / 100	\$0.082 100.00 / 100	\$0.245 100.00 / 100	\$0.238 100.00 / 100	\$0.142 100.00 / 100
Sol 6.1 / high	\$0.055 100.00 / 100	\$0.070 100.00 / 100	\$0.099 100.00 / 100	\$0.331 100.00 / 100	\$0.361 100.00 / 100	\$0.203 100.00 / 100
Sol 6.1 / xhigh	\$0.069 100.00 / 100	\$0.100 100.00 / 100	\$0.148 100.00 / 100	\$0.480 100.00 / 100	\$0.505 100.00 / 100	\$0.241 100.00 / 100
Sol 6.1 / max	\$0.118 100.00 / 100	\$0.117 100.00 / 100	\$0.146 100.00 / 100	\$0.509 100.00 / 100	\$0.591 100.00 / 100	\$0.267 100.00 / 100
Sol 6.1 / ultra	\$0.097 100.00 / 100	\$0.095 100.00 / 100	\$0.167 100.00 / 100	\$0.404 100.00 / 100	\$0.496 100.00 / 100	\$0.233 100.00 / 100

Asterisks mark partial cost telemetry. These are saved-code functional grades; completed acceptance is reported separately. Task-level agent times appear in the secondary time comparison.

Complex reasoning: behavioral evidence

Every small optimizer oracle and every MVCC check passed. Only one structured optimizer scale check failed.

Model / effort	Opt V	Opt C	Opt O	Opt S	MV Sem	MV Iso	MV Rec	MV V
Sol 5.6 / low	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 5.6 / medium	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 5.6 / high	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 5.6 / xhigh	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 5.6 / max	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 5.6 / ultra	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 6 / low	100.0	100.0	100.0	83.3	100.0	100.0	100.0	100.0
Sol 6 / medium	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 6 / high	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 6 / xhigh	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 6 / max	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 6 / ultra	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 6.1 / low	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 6.1 / medium	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 6.1 / high	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 6.1 / xhigh	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 6.1 / max	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 6.1 / ultra	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0

Optimizer: validation 20%, constraints 25%, optimality 35%, scale 20%. MVCC: semantics 25%, isolation 35%, recovery 25%, validation 15%. Values are category percentages; subcases fail their containing check.

The optimizer uses an independent exhaustive oracle on 64 seeded small problems per attempt, greedy traps, all tie rules and three structured scale fixtures. MVCC uses 480-step mixed transaction traces, 180-transaction replay traces, every checkpoint/log prefix, ABA, phantoms, write skew and failed-operation atomicity.

Scale failure: Sol 6 / Low, optimizer repetition 1, exceeded 8 seconds on the positive-bound fixture: 31 affordable projects and one individually infeasible high-value decoy. Its task score is 93.33; the paired task mean is 96.67. All its exact small-case answers passed.

A separate bounded replay reproduced the 8-second timeout. The positive control and Low repetition 2 completed the same fixture in about 0.20 and 0.22 seconds. The original grades were retained.

These tasks distinguish tested correctness and scaling behavior. Near-perfect outcomes across most settings leave too little separation to claim which model is generally smarter.

Expanded correctness and found defects

The first four tasks retain the same quality-v2.1 checks and scores. Completed solutions pass 920/922 expanded checks.

Model / effort	Intervals	TTL/LRU cache	Concurrent DAG	SQLite service
Sol 5.6 / low	100.00	94.64	100.00	100.00
Sol 5.6 / medium	100.00	100.00	100.00	100.00
Sol 5.6 / high	100.00	100.00	100.00	100.00
Sol 5.6 / xhigh	100.00	100.00	100.00	100.00
Sol 5.6 / max	100.00	100.00	100.00	100.00
Sol 5.6 / ultra	100.00	100.00	100.00	100.00
Sol 6 / low	100.00	94.64	100.00	100.00
Sol 6 / medium	100.00	100.00	100.00	100.00
Sol 6 / high	100.00	100.00	100.00	100.00
Sol 6 / xhigh	100.00	100.00	100.00	100.00
Sol 6 / max	100.00	100.00	100.00	N/A
Sol 6 / ultra	N/A	100.00	100.00	100.00
Sol 6.1 / low	100.00	100.00	100.00	100.00
Sol 6.1 / medium	100.00	100.00	100.00	100.00
Sol 6.1 / high	100.00	100.00	100.00	100.00
Sol 6.1 / xhigh	100.00	100.00	100.00	100.00
Sol 6.1 / max	100.00	100.00	100.00	100.00
Sol 6.1 / ultra	100.00	100.00	100.00	100.00

Boundary defects: Sol 5.6 / Low r2 and Sol 6 / Low r1 cache constructors fail on a finite integer TTL of $10^{**}400$ with an integer clock. Applying `math.isfinite` to that integer raises `OverflowError`. Their original acceptance tests passed.

Expanded tests add 1,200 random unions, cache model traces, DAG failure graphs and concurrency barriers, plus reservation stock-model steps, races and persistence/error cases. Properties/boundaries/concurrency weigh 40/30/30, renormalized when a task lacks a category.

N/A is incomplete completed-code coverage. The provider-failed stub and saved timeout code remain in raw evidence; they are excluded from completed-code means. The new tasks use their own frozen behavioral rubric, without retroactively changing old scores.

Do the added tests detect defects?

Per-task mutation sensitivity, averaged across two completed candidates. N/A shows usable-score coverage for that pair.

Model / effort	Intervals	Cache	DAG	Service	Optimizer	MVCC
Sol 5.6 / low	50%	50%	70%	58%	0%	0%
Sol 5.6 / medium	100%	60%	70%	0%	0%	50%
Sol 5.6 / high	100%	60%	90%	0%	N/A (1/2)	42%
Sol 5.6 / xhigh	50%	60%	80%	0%	90%	42%
Sol 5.6 / max	50%	60%	100%	0%	90%	67%
Sol 5.6 / ultra	50%	60%	N/A (1/2)	0%	N/A (1/2)	58%
Sol 6 / low	0%	30%	70%	0%	40%	25%
Sol 6 / medium	88%	60%	100%	N/A (1/2)	N/A (0/2)	75%
Sol 6 / high	88%	60%	100%	N/A (0/2)	100%	67%
Sol 6 / xhigh	100%	N/A (0/2)	100%	N/A (0/2)	90%	83%
Sol 6 / max	100%	N/A (1/2)	100%	N/A (0/2)	90%	N/A (1/2)
Sol 6 / ultra	N/A (1/2)	70%	100%	N/A (0/2)	N/A (1/2)	83%
Sol 6.1 / low	100%	70%	80%	100%	N/A (1/2)	75%
Sol 6.1 / medium	100%	80%	100%	N/A (0/2)	N/A (0/2)	92%
Sol 6.1 / high	100%	80%	100%	N/A (0/2)	N/A (0/2)	100%
Sol 6.1 / xhigh	100%	80%	100%	N/A (0/2)	N/A (0/2)	100%
Sol 6.1 / max	100%	80%	100%	N/A (0/2)	N/A (0/2)	100%
Sol 6.1 / ultra	100%	90%	100%	N/A (0/2)	N/A (0/2)	100%

175/214 completed candidates have usable scores; 29 have no recognized added tests. On the complex tasks, 55/72 scores are usable and 10 candidates have no recognized added tests.

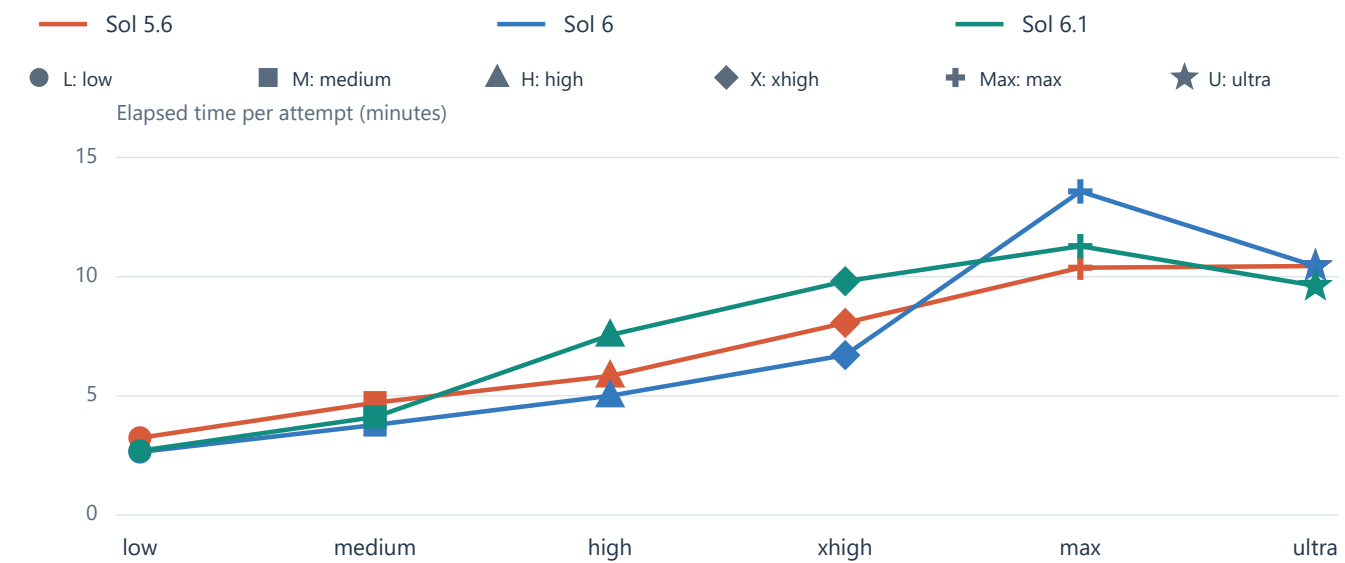
Discovery uses unittest-compatible tests/test*.py, excluding supplied test_public.py. Tests must pass on both candidate and positive control. No added tests score zero only for this criterion. Import errors, skips, timeouts and reference incompatibility are inconclusive.

Optimizer test strength is unavailable for 16/36 candidates: twelve positive-control test runs hit the 30-second timeout, three encounter import/execution issues, and one mutant run is inconclusive. These evaluator limitations particularly affect Sol 6.1 optimizer coverage and do not establish a candidate-code defect.

Fixed sets: 4 interval, 5 cache, 5 DAG, 6 service, 5 optimizer and 6 MVCC defects. The original four-task classifier detects an executed test failure or error after the reference passed; loader failures/timeouts are inconclusive. The complex-task classifier counts assertion failures only and treats execution errors/hangs as unavailable. Interpret sensitivity within each task.

Agent time: a secondary comparison

Time remains useful for workflow planning, separate from the primary money-versus-quality comparison. The top curve shows mean elapsed agent minutes; the table reports each task in seconds.



Model / effort	Intervals	Cache	DAG	Service	Optimizer	MVCC
Sol 5.6 / low	69	108	125	280	350	233
Sol 5.6 / medium	91	130	165	399	621	290
Sol 5.6 / high	108	148	198	531	755	360
Sol 5.6 / xhigh	147	141	215	692	1,298	410
Sol 5.6 / max	179	212	305	908	1,584	548
Sol 5.6 / ultra	151	225	314	941	1,524	606
Sol 6 / low	68	78	110	253	240	204
Sol 6 / medium	88	112	145	299	490	226
Sol 6 / high	77	119	168	505	683	249
Sol 6 / xhigh	194	242	209	666	762	343
Sol 6 / max	219	275	301	2,380	1,220	492
Sol 6 / ultra	158	215	215	1,230	1,493	447
Sol 6.1 / low	72	96	123	272	221	184
Sol 6.1 / medium	81	105	152	413	490	240
Sol 6.1 / high	165	166	268	871	863	386
Sol 6.1 / xhigh	121	158	325	1,028	1,394	506
Sol 6.1 / max	230	224	385	1,096	1,580	545
Sol 6.1 / ultra	134	176	420	1,068	1,198	466

Elapsed agent time includes reasoning, commands, self-tests and waits during the turn. Setup, snapshotting and external grading are excluded. Task limits are 10 / 20 / 40 / 40 / 40 / 40 minutes; interrupted durations remain included.

Generated-code runtime and allocation

Existing quality-v2.1 runtime / Python-allocation measurements for the first four tasks. Each cell is milliseconds / KiB.

Model / effort	Intervals	TTL/LRU cache	Concurrent DAG	SQLite service
Sol 5.6 / low	17.90 / 4,336	34.68 / 104	54.16 / 953	515.90 / 4
Sol 5.6 / medium	16.34 / 4,454	34.95 / 104	61.10 / 901	478.92 / 4
Sol 5.6 / high	18.84 / 4,454	35.75 / 104	63.61 / 870	533.39 / 4
Sol 5.6 / xhigh	17.02 / 4,454	35.53 / 104	60.55 / 877	553.25 / 5
Sol 5.6 / max	16.83 / 4,336	34.36 / 104	56.91 / 755	648.41 / 5
Sol 5.6 / ultra	17.61 / 4,336	33.78 / 104	53.81 / 863	611.56 / 4
Sol 6 / low	13.95 / 4,336	39.05 / 112	54.40 / 802	537.83 / 4
Sol 6 / medium	15.54 / 4,336	19.90 / 112	58.85 / 806	543.75 / 4
Sol 6 / high	15.16 / 4,336	40.23 / 112	55.15 / 730	693.05 / 4
Sol 6 / xhigh	16.97 / 4,336	40.44 / 112	51.28 / 813	622.31 / 4
Sol 6 / max	16.25 / 4,336	38.47 / 112	57.80 / 844	N/A
Sol 6 / ultra	N/A	36.34 / 108	57.56 / 838	504.32 / 4
Sol 6.1 / low	17.07 / 4,336	33.74 / 104	77.32 / 781	529.36 / 5
Sol 6.1 / medium	17.29 / 4,336	34.85 / 104	71.42 / 834	861.48 / 5
Sol 6.1 / high	16.68 / 4,336	35.37 / 104	66.49 / 806	555.97 / 5
Sol 6.1 / xhigh	16.82 / 4,336	35.33 / 104	68.93 / 730	468.35 / 5
Sol 6.1 / max	17.24 / 4,336	38.25 / 104	55.46 / 704	511.43 / 5
Sol 6.1 / ultra	17.30 / 4,336	34.98 / 104	52.31 / 705	467.98 / 5

One warm-up, then the median of three sequential runs. Workloads: 30,000 intervals; cache capacity 1,000 with 1,000 writes and 3,000 reads; 1,500 DAG nodes; 40 reservation cycles. Allocation is a separate tracemalloc measurement, not process RSS or native/SQLite memory.

These values describe execution after generation, separate from agent completion time. The complex tasks have their scored scale/trace checks; equivalent runtime and memory profiles were not collected for them. Small differences can reflect scheduling noise.

Repetition spread

Median second-to-first ratios across six task pairs. Two observations per pair support descriptive spread, not confidence intervals.

Model / effort	USD r2 / r1	Time r2 / r1	Largest time spread
Sol 5.6 / low	0.82	0.97	1.42x
Sol 5.6 / medium	0.82	0.98	1.43x
Sol 5.6 / high	1.11	1.09	1.47x
Sol 5.6 / xhigh	0.92	0.95	1.52x
Sol 5.6 / max	0.68	0.82	2.21x
Sol 5.6 / ultra	0.96	0.98	1.20x
Sol 6 / low	0.68	0.77	1.57x
Sol 6 / medium	0.93	0.95	1.45x
Sol 6 / high	1.04	1.08	1.22x
Sol 6 / xhigh	0.97	1.03	2.24x
Sol 6 / max	N/A	1.00	1.31x
Sol 6 / ultra	N/A	1.05	1.88x
Sol 6.1 / low	0.96	1.10	1.25x
Sol 6.1 / medium	0.94	0.92	1.87x
Sol 6.1 / high	0.97	0.78	1.68x
Sol 6.1 / xhigh	0.81	1.00	1.80x
Sol 6.1 / max	1.12	0.97	1.57x
Sol 6.1 / ultra	0.87	0.90	1.63x

A ratio above 1 means repetition two used more money/time. The last column is the largest max/min time ratio among that setting's six pairs, including interruptions. Server load and tool behavior can affect elapsed time.

Paid attempts are sequential, with model/effort order interleaved and reversed inside each task in repetition two. There are no automatic retries, model fallbacks or post-grading feedback.

Code quality and design review

Behavior, test strength and execution efficiency are reported separately. Manual rubric scoring remains pending.

Evidence layer	Scope	Interpretation
Main functional score	All six tasks	Equal task weights; fixed category weights for the complex tasks.
Expanded correctness	First four tasks	Frozen properties, boundaries and concurrency checks.
Own-test strength	All six tasks	Sensitivity to fixed per-task contract mutations.
Runtime / memory	First four tasks	Descriptive generated-code measurements, unscored.
Manual design rubric	Pending	Clarity, design and failure maintenance, with source/line evidence.

Manual rubric

Area	Evidence a reviewer should inspect
Clarity	Intent, names, coherent control flow, explanations and useful tests.
Design	Interfaces, responsibilities, cohesion, duplication and contract fit.
Failure maintenance	Error handling, cleanup, transactions, concurrency and safe changes.

The existing four-task quality formula uses 30% original correctness, 35% expanded correctness, 20% own-test strength and 15% manual review. Its normalized 85-point automatic evidence is not a full quality score. It is preserved for those tasks and is not extrapolated to the added tasks.

Static line/function counts are review signals, not grades. Tests tied to private implementation details can be valid for the candidate while yielding inconclusive cross-implementation mutation evidence.

Method, token prices and reproducibility

Each attempt uses one turn, a fresh session/workspace, the exact requested model/effort and Standard speed. Task prompt hashes are identical within each task across all 36 attempts. SHA-256 manifests protect the prepared inputs and saved candidate code.

Model	Input	Cached input	Cache write	Output
Sol 5.6	\$4.00	\$0.40	\$5.00	\$20.00
Sol 6	\$2.00	\$0.20	\$2.50	\$10.00
Sol 6.1	\$2.00	\$0.10	\$2.50	\$10.00

Rates are USD per one million tokens, frozen on 30 Sep 2026 and verified on 1 Oct. The same schedule is used throughout for comparability. This report does not assert a newly checked current price. Ultra is a verified Codex effort setting; the USD calculation does not imply an API Ultra option.

Uncached input, cached reads, cache writes and output are priced without double counting. Reasoning tokens are part of output. Per-request input above 272,000 tokens applies the frozen 2x input/cache and 1.5x output multipliers. Hosted-tool fees and regional premiums are excluded.

Exact token estimates: 214/216. Observed total is at least \$87.59; two partial-usage attempts prevent a complete total. The two complex tasks have exact observed token cost \$41.81 across 72 attempts.

Secondary token-based credit estimate: 2,189.95 observed credits, including partial usage. Shared account balance changes do not identify billed credits for an individual run.

Pinned runtime: codex-cli 0.159.0; Node v22.18.0; Python 3.14.0 on Windows. Shell/tool recovery is included in agent time. Read isolation is instruction-based; automated command scans are diagnostics.

Sources and evidence

Sol 5.6 API rates | Sol 6 API rates | Sol 6.1 API rates
Codex credits | Codex app-server

Public evidence: results/results.json, quality-v2.1.json, complex-test-strength.json, report-audit.json and provenance.json; candidates/ and per-attempt prompt/final evidence. The original local audit verified all 216 snapshots/settings/turns and price calculations. Raw RPC/session logs are not distributed. This revision changes presentation, with no new inference or rescoring.

Public results SHA-256: b20d615c1e4334911db470e27fc163f31efa3a14e32161cf3a57e2aea0ab5fbe
Original source identities and the unchanged 4 Oct report are recorded in results/provenance.json. Current report revision: 5 Oct 2026.